BÖLÜM 9 / MODULE 9

FluxCD ile Modern GitOps – Tek Gerçek Kaynak Git!

Modern GitOps with FluxCD – Git as the Single Source of Truth

Canlı sunucuya el sürmeye son: GitOps felsefesi, FluxCD mimarisi, GitHub bootstrap kurulumu, otomatik imaj güncellemesi ve deklaratif Helm yönetimi.

Eliminating manual cluster tinkering: GitOps principles, FluxCD architecture, GitHub bootstrapping, automated image releases, and declarative Helm management.

49

BÖLÜM: FLUXCD ILE MODERN GITOPS – TEK GERÇEK KAYNAK GIT! | MODULE: MODERN GITOPS WITH FLUXCD – GIT AS THE SINGLE SOURCE OF TRUTH

GitOps Nedir? Neden `kubectl apply` Artık Yasak?

What is GitOps? Why Manual `kubectl apply` is an Anti-Pattern

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Bir şirkette 10 farklı yazılımcının canlı sunucuya bağlanıp kafasına göre `kubectl apply` yapması, bir restorandaki 10 aşçının aynı çorbaya gizlice farklı tuzlar atmasına benzer (Sonuç tam bir zehirlenme faciası!). Kim ne değiştirdi, dün ne çalışıyordu kimse bilmez! GitOps'ta ise kural kesindir: Canlı sunucuya KİMSE el süremez! Tek gerçek kaynak Git deposudur. Bir şey değişecekse Git'e commit atılır, kümenin içindeki bekçi robot (Flux) değişikliği çeker!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Ten engineers SSH'ing into production to fire off manual `kubectl apply` commands is like ten rogue chefs secretly dumping different spices into the soup without telling each other (Total disaster!). Nobody knows who broke production! GitOps establishes an unbreakable law: Git is the Single Source of Truth. Nobody has cluster write access. You commit changes to Git, and a dedicated cluster agent (Flux) pulls and reconciles them!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Geleneksel CI/CD sistemleri 'Push' (İtme) modeliyle çalışır: Jenkins veya GitLab sunucusu doğrudan K8s üretim kümesinin `cluster-admin` şifresine sahiptir ve dışarıdan kümeye bağlanıp değişiklik basar (Büyük güvenlik riski!). GitOps ise 'Pull' (Çekme) modeliyle çalışır: K8s kümesinin dışarıya hiçbir portu veya yönetim yetkisi açılmaz. Küme içinde çalışan Flux denetleyicisi, dışarıdaki Git deposunu periyodik olarak yoklar (poll) ve kümedeki 'yapılandırma sapmalarını' (configuration drift) otomatik onarır.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Traditional CI/CD relies on an insecure 'Push' model: external build runners possess broad `cluster-admin` credentials to inject updates into production. GitOps inverts this via a zero-trust 'Pull' architecture: the cluster exposes no inbound management ports. Instead, an in-cluster agent (Flux) continuously pulls from Git, comparing Git's declared state against live cluster reality and reversing any unauthorized configuration drift.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# Geleneksel Tehlikeli Yöntem (Mühendis terminalinden canlıya müdahale):
kubectl apply -f deployment.yaml # YASAK! İz bırakmaz, geri alınamaz!

# GitOps Yolu (Güvenli, Denetlenebilir, Kurumsal Standart):
git add clusters/production/coffee-app/
git commit -m "feat(coffee): upgrade spring-boot to v1.2.0"
git push origin main

# Küme içindeki Flux robotu değişikliği saniyeler içinde otomatik çeker!
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

GitOps kurulu bir kümede bir mühendis gidip canlıda `kubectl edit deployment` ile değişiklik yaparsa ne olur? Flux bir dakika içinde bu değişikliği yakalar, 'Git deposunda böyle bir şey yazmıyor!' diyerek değişikliği ezer ve sistemi Git'teki haline geri döndürür!

🇬🇧 WATCH OUT

If a rogue engineer modifies a live pod via `kubectl edit`, Flux detects the drift within 60 seconds, nukes the manual edit, and enforces the exact state declared in Git.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Git tüm altyapının ve yazılımların tek resmi versiyon kütüğüdür.
- Çekme (Pull) modeli sayesinde küme erişim anahtarları dışarıya sızmaz.
- Tüm dağıtım geçmişi `git log` üzerinden kimin ne yaptığıyla açıkça izlenir.

- Git is the immutable, auditable single source of truth for the entire cluster.
- The Pull model eliminates the security hazard of storing cluster tokens in external CI.
- Complete auditability and compliance: every cluster change corresponds to a signed commit.

50

BÖLÜM: FLUXCD ILE MODERN GITOPS – TEK GERÇEK KAYNAK GIT! | MODULE: MODERN GITOPS WITH FLUXCD – GIT AS THE SINGLE SOURCE OF TRUTH

FluxCD Mimarisi: 4 Silahşörler Kontrolcüleri

FluxCD Architecture: The Four Musketeer Controllers

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Flux devasa, hantal tek bir program değildir; her biri kendi işinde uzmanlaşmış 4 süper kahraman robottur: 1) Source Controller: Gözlerini kırpmadan Git deposunu ve Docker registry'yi izleyen gözcü. 2) Kustomize Controller: Çekilen YAML'ları K8s kümesine uygulayan usta işçi. 3) Helm Controller: Dışarıdan paket indiren kargo şefi. 4) Notification Controller: Bir şey olduğunda Slack veya Teams'e haber uçuran haberci!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Flux is not a monolithic bloated daemon; it is a modular league of 4 specialized micro-controllers: 1) Source Controller: The tireless scout watching Git repos and OCI registries. 2) Kustomize Controller: The builder reconciling raw manifests into the cluster. 3) Helm Controller: The logistics manager orchestrating Helm chart lifecycles. 4) Notification Controller: The town crier alerting Slack/Teams when events occur!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Flux v2 (GitOps Toolkit) mimarisi Kubernetes Controller Runtime prensiplerine tam uyumludur: 1) `source-controller`: Git depolarını, OCI imajlarını, Helm repo'larını klonlar ve bir gzip tarball olarak küme içi belleğe sunar. 2) `kustomize-controller`: Bu paketi alır, Kustomize yamalarını (overlays) çözer, K8s API'sine uygular ve `prune: true` ile Git'ten silinen kaynakları kümeden de siler. 3) `helm-controller`: Helm paketlerinin sürümlerini ve yükseltmelerini bildirici yönetir. 4) `notification-controller`: Webhook'ları kabul eder.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

The Flux GitOps Toolkit is a composable set of specialized controllers: 1) `source-controller` clones Git/Helm/Bucket repositories and serves them as localized tarball artifacts. 2) `kustomize-controller` evaluates manifests via Kustomize engines, applies resources to the API Server, and garbage collects orphaned resources when deleted from Git (`prune: true`). 3) `helm-controller` manages Helm chart releases declaratively. 4) `notification-controller` manages outbound notifications and inbound webhooks.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# Flux Kontrolcülerinin Kümedeki Durumunu Sorgula:
flux check

# Çıktı:
# ► checking controllers
# ✔ source-controller: deployment ready
# ✔ kustomize-controller: deployment ready
# ✔ helm-controller: deployment ready
# ✔ notification-controller: deployment ready
# ✔ all checks passed
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Eski Flux v1 tek parçaydı (monolith). Modern Flux v2 ise modüler 'GitOps Toolkit' üzerine kuruludur. Eski v1 dokümanlarındaki `fluxctl` komutlarını kullanmayın; daima modern `flux` CLI'ını kullanın!

🇬🇧 WATCH OUT

Never use legacy Flux v1 tutorials or the deprecated `fluxctl` CLI. Modern Flux v2 is entirely re-architected around the GitOps Toolkit controller family.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Flux modüler yapısı sayesinde sadece ihtiyaç duyulan kontrolcüler kurulabilir.
- Kustomize Controller otomatik çöp toplayıcı (Garbage Collector) gibi çalışır.
- Notification Controller ile dağıtım durumları Slack veya Discord'a bağlanır.

- Micro-controller architecture allows adopting only the controllers you actually need.
- Kustomize controller's `prune: true` automatically purges deleted resources cleanly.
- Notification controller routes release events directly to corporate communication channels.

51

BÖLÜM: FLUXCD ILE MODERN GITOPS – TEK GERÇEK KAYNAK GIT! | MODULE: MODERN GITOPS WITH FLUXCD – GIT AS THE SINGLE SOURCE OF TRUTH

Flux Bootstrap: K8s Kümesi ile GitHub Deposunu Nikahlama

Flux Bootstrap: Marrying the K8s Cluster to a GitHub Repo

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Eskiden GitOps kurmak haftalarca süren sertifika ve konfigürasyon eziyetiydi. Flux Bootstrap tek bir sihirli nikah memurudur! Terminale tek bir komut yazarsınız (`flux bootstrap github`): K8s kümenize gider Flux'ı kurar, GitHub hesabınızda yepyeni bir repo açar, içine şifreli SSH anahtarlarını yerleştirir ve ikisini sonsuza dek birbirine bağlar!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Setting up GitOps used to require days of manual SSH key juggling. Flux Bootstrap is a one-line automated marriage officiant! You run a single command (`flux bootstrap github`): it installs Flux on your cluster, creates the Git repository on your GitHub account, configures secure deploy keys, and binds them together in holy matrimony!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

`flux bootstrap` komutu hem emirsel başlatıcı hem de bildirici bir döngüdür. Arka planda: 1) GitHub API'sini kullanarak belirtilen organizasyon veya kullanıcı altında Git reposunu oluşturur. 2) Kümede Flux manifestolarını ayağa kaldırır. 3) Bir SSH Deploy Key çifti üretir; genel anahtarı GitHub reposuna 'read/write' yetkisiyle ekler, özel anahtarı ise K8s `flux-system` namespace'ine Secret olarak kilitler. 4) Flux'ın kendi manifestolarını Git'e commit atar; böylece Flux kendi kendini bile GitOps ile günceller!

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

The `flux bootstrap` command is idempotent and self-referential: 1) It provisions the target Git repository via the GitHub REST API, 2) Generates a dedicated SSH keypair, installing the public key as a GitHub Deploy Key and the private key as a Kubernetes Secret, 3) Commits its own component manifests into Git (`flux-system`), allowing Flux to upgrade itself via GitOps.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```
# GitHub Kişisel Erişim Belirteci (Token) Belirle:
export GITHUB_TOKEN="ghp_yourPersonalAccessTokenHere"

# Tek Komutla Flux Bootstrap Kurulumu:
flux bootstrap github \
  --owner=my-org \
  --repository=k8s-gitops-fleet \
  --branch=main \
  --path=clusters/production \
  --personal
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Oluşturduğunuz GitHub Token'ın `repo` izinlerine sahip olduğundan emin olun! Token izinleri yetersizse Bootstrap işlemi reponun açılması veya deploy key eklenmesi aşamasında hata verir.

🇬🇧 WATCH OUT

Your GitHub Personal Access Token requires full `repo` scopes! Insufficient token permissions will fail during deploy key or webhook injection.

🎯 Özet Çıkarımlar

🎯 Key Takeaways

- Bootstrap işlemi küme ile Git deposu arasındaki güvenli köprüyü kurar.
- Flux kendi kendini yöneten (self-managing) bir yapıya kavuşur.
- Kümenin tüm tanımı Git üzerindeki `clusters/production` klasöründe saklanır.

- Bootstrap establishes the cryptographically secured bridge between Git and cluster.
- Flux manages its own version and configuration via GitOps once bootstrapped.
- Cluster state lives transparently inside the version-controlled `clusters/` directory.

52

BÖLÜM: FLUXCD ILE MODERN GITOPS – TEK GERÇEK KAYNAK GIT! | MODULE: MODERN GITOPS WITH FLUXCD – GIT AS THE SINGLE SOURCE OF TRUTH

GitRepository ve Kustomization ile Spring Boot Dağıtımı

Deploying Spring Boot via GitRepository & Kustomization

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Bakkala sipariş verirken iki şeye ihtiyaç vardır: 1) Hangi bakkala gideceğiniz (`GitRepository`: 'Git şu GitHub deposuna bak!'), 2) Bakkaldan ne alacağınız ve eve nasıl yerleştireceğiniz (`Kustomization`: 'Deponun içindeki `coffee-app` klasörünü al, K8s kümesindeki `coffee-shop` ortamına kur ve her 5 dakikada bir kontrol et!').

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Ordering groceries requires two distinct instructions: 1) Where is the grocery store? (`GitRepository`: 'Track this GitHub repository branch!'), 2) What items to fetch and where to put them? (`Kustomization`: 'Extract the manifests inside `./apps/coffee-app`, apply them into the `coffee-shop` namespace, and check every 5 minutes!').

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

1) `GitRepository`: Source Controller'a hangi Git URL'sini, hangi branch'i (`ref.branch: main`) ve hangi kontrol aralığını (`interval: 1m`) izleyeceğini söyler. 2) `Kustomization`: Kustomize Controller'a bu repodaki hangi dizinin (`path: ./clusters/production/coffee-app`) hangi namespace'e uygulanacağını belirtir. `prune: true` parametresiyle Git'ten bir YAML silindiğinde K8s kümesinden de otomatik silinmesi sağlanır.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

1) `GitRepository` directs `source-controller` to pull from a Git endpoint, branch, and polling interval, producing an immutable tarball. 2) `Kustomization` instructs `kustomize-controller` to evaluate that tarball at a designated sub-path, target namespace, and interval. Enabling `prune: true` activates automated garbage collection of resources removed from Git.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```yaml
# spring-boot-gitops.yaml
apiVersion: source.toolkit.fluxcd.io/v1
kind: GitRepository
metadata:
  name: fleet-repo
  namespace: flux-system
spec:
  interval: 1m
  url: https://github.com/my-org/k8s-gitops-fleet
  ref:
    branch: main
---
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata:
  name: coffee-app-kustomization
  namespace: flux-system
spec:
  interval: 5m
  targetNamespace: coffee-shop
  sourceRef:
    kind: GitRepository
    name: fleet-repo
  path: ./clusters/production/coffee-app
  prune: true
  wait: true # Pod'lar ayağa kalkana kadar dağıtımı bekle!
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

`prune: true` parametresi ÇOK GÜÇLÜDÜR! Eğer yanlışlıkla Git deposundan bir YAML dosyasını silip push ederseniz, Flux o servisi ve pod'ları canlı kümeden anında siler! Dikkatli kullanılmalıdır.

🇬🇧 WATCH OUT

`prune: true` is unforgiving! If you delete a manifest file from Git and commit, Flux will immediately delete the live workload from the cluster.

🎯 Özet Çıkarımlar

🎯 Key Takeaways

- GitRepository kaynak izleme görevini, Kustomization ise uygulama görevini üstlenir.
- `prune: true` ile yetim kalan (orphaned) eski kaynaklar otomatik temizlenir.
- `flux get kustomizations` komutuyla dağıtım sağlığı anlık olarak izlenir.

- GitRepository handles artifact pulling; Kustomization handles manifest application.
- `prune: true` guarantees complete synchronization by sweeping away orphaned resources.
- Inspect continuous delivery status via `flux get kustomizations`.

53

BÖLÜM: FLUXCD ILE MODERN GITOPS – TEK GERÇEK KAYNAK GIT! | MODULE: MODERN GITOPS WITH FLUXCD – GIT AS THE SINGLE SOURCE OF TRUTH

Otomatik İmaj Güncellemesi (ImageUpdateAutomation)

Automated Image Updates: Flux Auto-Commit Magic

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Gözlerinize inanamayacağınız bir otomasyon! GitHub Actions yeni bir Spring Boot Docker imajı derleyip registry'ye yüklediğinde (örn: `v1.0.5`), Flux bunu tespit eder. Ardından Flux KENDİ KENDİNE Git deponuza bir commit atar: 'Ben yeni imajı gördüm, manifestoyu v1.0.5 yaptım!' der, push eder ve küme güncellenir! Hiçbir insana ihtiyaç kalmaz!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Pure hands-free engineering magic! When your CI pipeline bakes and pushes a new Spring Boot container image (e.g. `v1.0.5`), Flux sniffs it out. Then Flux ITSELF writes a Git commit to your repository: 'Updated coffee-app image to v1.0.5', pushes the commit, and rolls out the release! Zero human clicks required!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Flux Image Automation üç özel CRD ile çalışır: 1) `ImageRepository`: Docker Registry'yi (GHCR, Quay) tarayarak yeni imaj etiketlerini (tags) keşfeder. 2) `ImagePolicy`: Semantik Sürümleme (SemVer: `1.0.x` veya `^1.0.0`) kurallarına göre en güncel stabil sürümü seçer. 3) `ImageUpdateAutomation`: Git deposundaki YAML dosyasında `# {'$imagepolicy': 'flux-system:coffee-policy'}` işaretçisini bulur, etiketi günceller ve Git'e otomatik yazar.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Flux Image Automation orchestrates 3 CRDs: 1) `ImageRepository` scans container registries for new OCI tags. 2) `ImagePolicy` resolves the newest tag using SemVer rules (`range: >=1.0.0 <2.0.0`). 3) `ImageUpdateAutomation` locates setter comments in YAML manifests (`# {"$imagepolicy": ...}`), updates the image tag in-place, and pushes a signed Git commit back to the repo.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# deployment.yaml içindeki Flux Sihirli Yorum Satırı:
spec:
  containers:
    - name: coffee-container
      # Flux bu satırı otomatik güncelleyip Git'e commit atar:
      image: ghcr.io/eli5-k8s/spring-boot-k8s-eli5:1.0.0 # {"$imagepolicy": "flux-
system:coffee-policy"}

# image-policy.yaml
apiVersion: image.toolkit.fluxcd.io/v1beta2
kind: ImagePolicy
metadata:
  name: coffee-policy
  namespace: flux-system
spec:
  imageRepositoryRef:
    name: coffee-image-repo
  policy:
    semver:
      range: '^1.0.0' # 1.x ailesindeki en yeni yamayı otomatik al!
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

ImageUpdateAutomation'ın çalışabilmesi için Flux'ın GitHub deposuna 'YAZMA' (write/push) yetkisinin olması gerekir. Salt-okunur (read-only) deploy key ile bu özellik çalışmaz!

🇬🇧 WATCH OUT

ImageUpdateAutomation requires write permissions to your Git repository! A read-only deploy key will fail when Flux attempts to push automated commits.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Yeni Docker imajları hiçbir manuel müdahale olmadan anında canlıya alınır.
- SemVer politikaları sayesinde sadece test edilmiş stabil yamalar alınır.
- Git deposu her zaman canlıda çalışan gerçek imaj etiketini tam olarak yansıtır.

- Container releases occur automatically without manual manifest editing toil.
- Semantic versioning policies prevent accidental promotion of untested major releases.
- Git commits ensure that your repository always mirrors what is live in production.

54

BÖLÜM: FLUXCD ILE MODERN GITOPS – TEK GERÇEK KAYNAK GIT! | MODULE: MODERN GITOPS WITH FLUXCD – GIT AS THE SINGLE SOURCE OF TRUTH

Helm Controller: Helm Paketlerini GitOps ile Yönetme

Declarative Helm Management with Flux HelmReleases

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Helm, Kubernetes dünyasının App Store paket yöneticisidir (npm veya pip gibi). Normalde terminalden `helm install postgresql ...` yazarsınız ama bu GitOps kuralını bozar! Flux'ın `HelmRelease` nesnesiyle deriz ki: 'Bitnami PostgreSQL paketini al, şu şifreleri ver, kur ve güncel tut!' Böylece devasa hazır yazılımları tek bir temiz YAML ile GitOps'a dahil ederiz!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Helm is the package manager for Kubernetes (like npm or pip). Normally developers run imperative `helm install postgresql ...` commands, violating GitOps! With Flux's `HelmRelease` resource, we declare: 'Fetch the Bitnami PostgreSQL chart, inject these custom passwords, and keep it healthy!' Sprawling third-party software stacks are tamed cleanly via GitOps!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Helm Controller iki kaynakla yönetilir: 1) `HelmRepository`: Helm chart'larının barındırıldığı depoyu (örn: `https://charts.bitnami.com/bitnami`) tanımlar. 2) `HelmRelease`: Belirli bir chart sürümünü (`version: '15.x'`), değerlerini (`values`) ve yükseltme stratejilerini tanımlar. Flux Helm Controller, chart'ın yükseltmesi başarısız olduğunda otomatik geri alma (`rollback`) yeteneğine sahiptir.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Flux manages Helm declaratively via: 1) `HelmRepository` points to Helm chart repositories or OCI registries. 2) `HelmRelease` declares chart versions, target namespaces, and customizable `values` overrides. The Helm Controller triggers native Helm library actions in-cluster, supporting automated rollbacks upon install/upgrade healthcheck failures.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```yaml
# postgres-helm-release.yaml
apiVersion: source.toolkit.fluxcd.io/v1
kind: HelmRepository
metadata:
  name: bitnami
  namespace: flux-system
spec:
  interval: 1h
  url: https://charts.bitnami.com/bitnami
---
apiVersion: helm.toolkit.fluxcd.io/v2
kind: HelmRelease
metadata:
  name: coffee-postgres
  namespace: coffee-shop
spec:
  interval: 15m
  chart:
    spec:
      chart: postgresql
      version: "15.5.x"
      sourceRef:
        kind: HelmRepository
        name: bitnami
        namespace: flux-system
  values:
    auth:
      database: coffeedb
      username: coffeeuser
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Helm paketlerinin `values` kısmında hassas veritabanı şifrelerini açık metin olarak yazmayın! Şifreler için `valuesFrom` ile bir Secret veya Mozilla SOPS ile şifrelenmiş değerler referans verilmelidir.

🇬🇧 WATCH OUT

Never hardcode cleartext passwords inside `HelmRelease.spec.values`! Reference external Kubernetes secrets using `valuesFrom` or Mozilla SOPS.

🎯 Özet Çıkarımlar

🎯 Key Takeaways

- HelmRelease Helm paketlerini tamamen bildirici ve GitOps uyumlu hale getirir.
- Yükseltme hatalarında otomatik rollback yaparak kesintiyi engeller.
- Tüm dış bağımlılıklar (Redis, Kafka, PostgreSQL) Git üzerinden versiyonlanır.

- HelmRelease makes Helm chart lifecycles 100% declarative and GitOps-native.
- Automated rollbacks recover from failed chart upgrades safely.
- Third-party dependencies are cleanly audited and versioned inside your fleet repo.